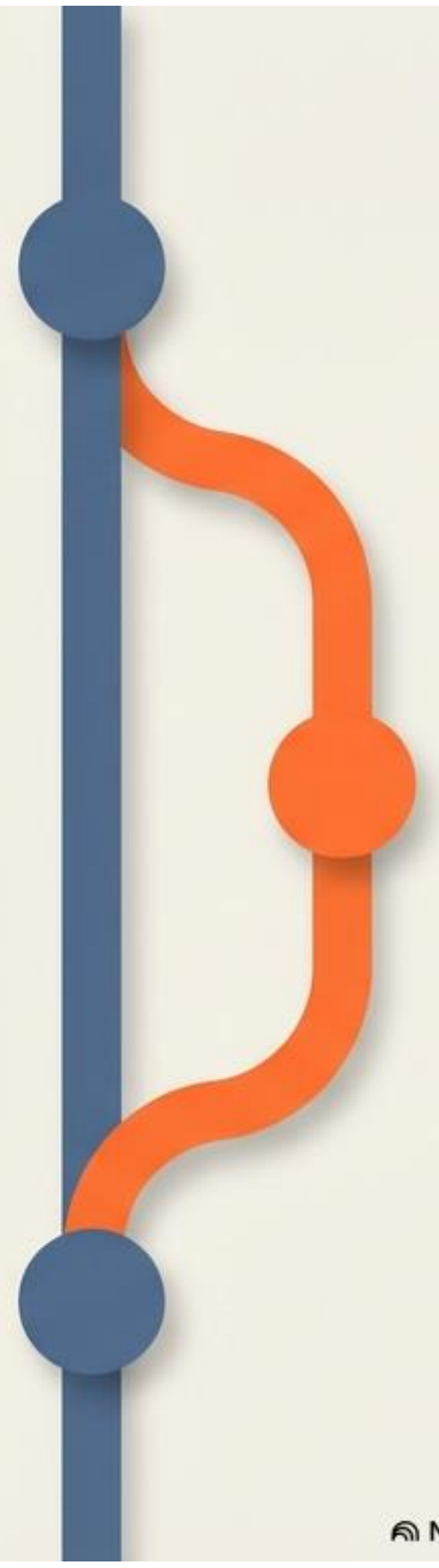


Version Control & Collaboration

A. prof. Elsayed Mahdy

Mastering Git for AI Projects

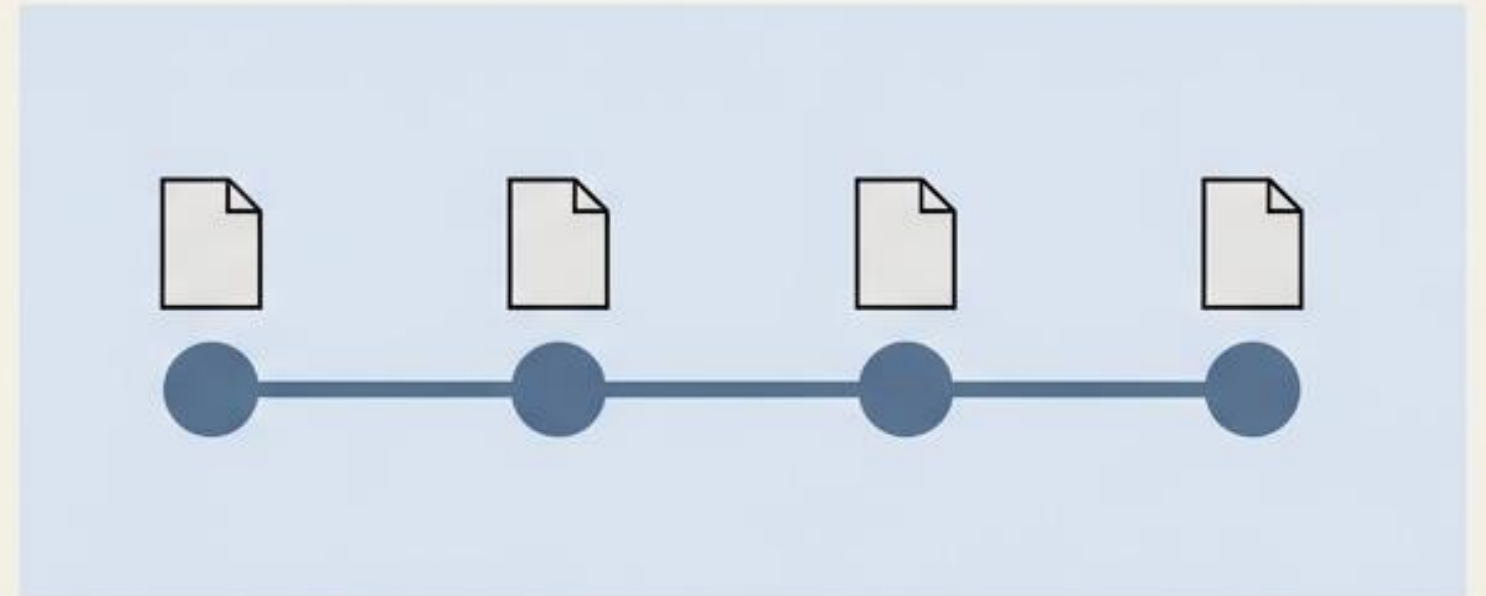
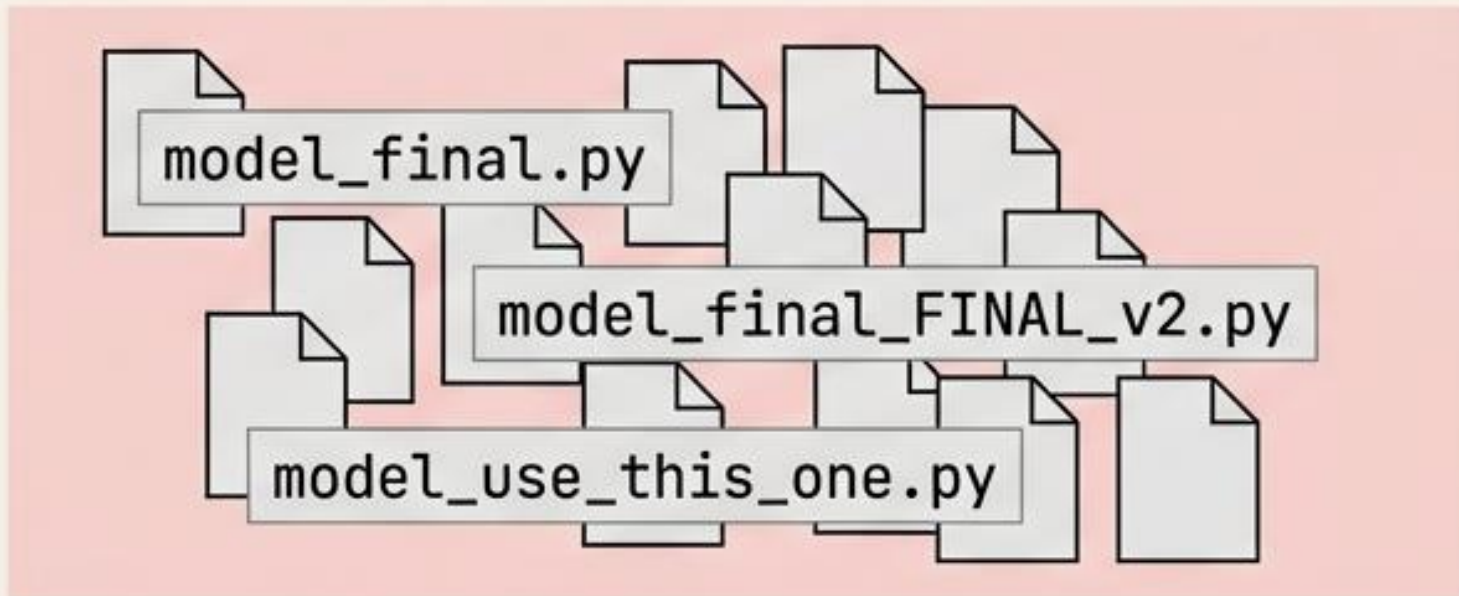
A complete tutorial on version control and team collaboration.



Moving from chaos to complete control

AI projects move fast—data shifts, features evolve, and models change constantly. Git provides the infrastructure to:

- Track every single change (maintain a complete history)
- Undo mistakes safely and reliably
- Collaborate simultaneously without overwriting files
- Guarantee reproducibility by knowing exactly which commit produced which model



The core mental model of version control



Repo: The specific project tracked by Git.



Commit: A saved snapshot of your work, accompanied by a descriptive message.



Branch: A separate, isolated workspace dedicated to a new feature or fix.



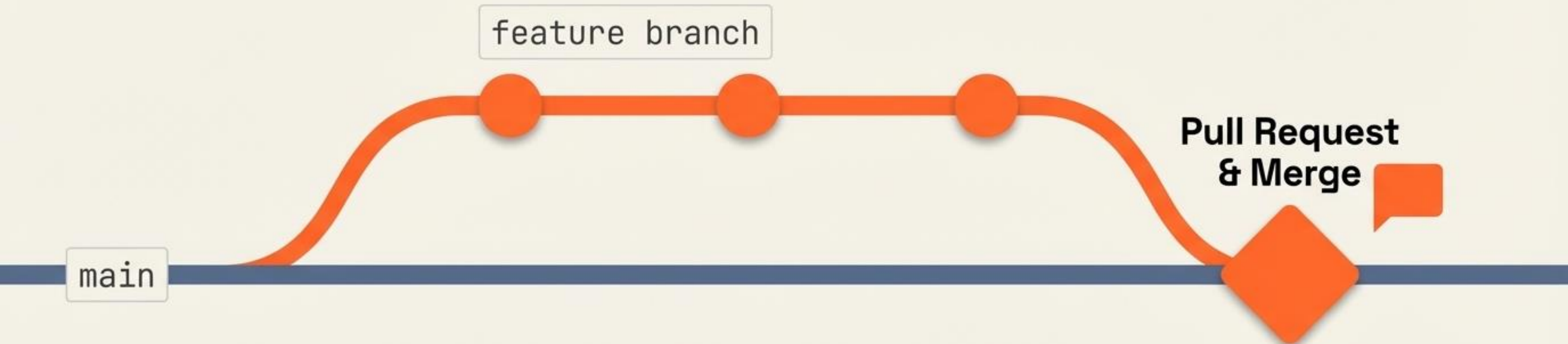
Remote (GitHub): The shared online copy of the repository.



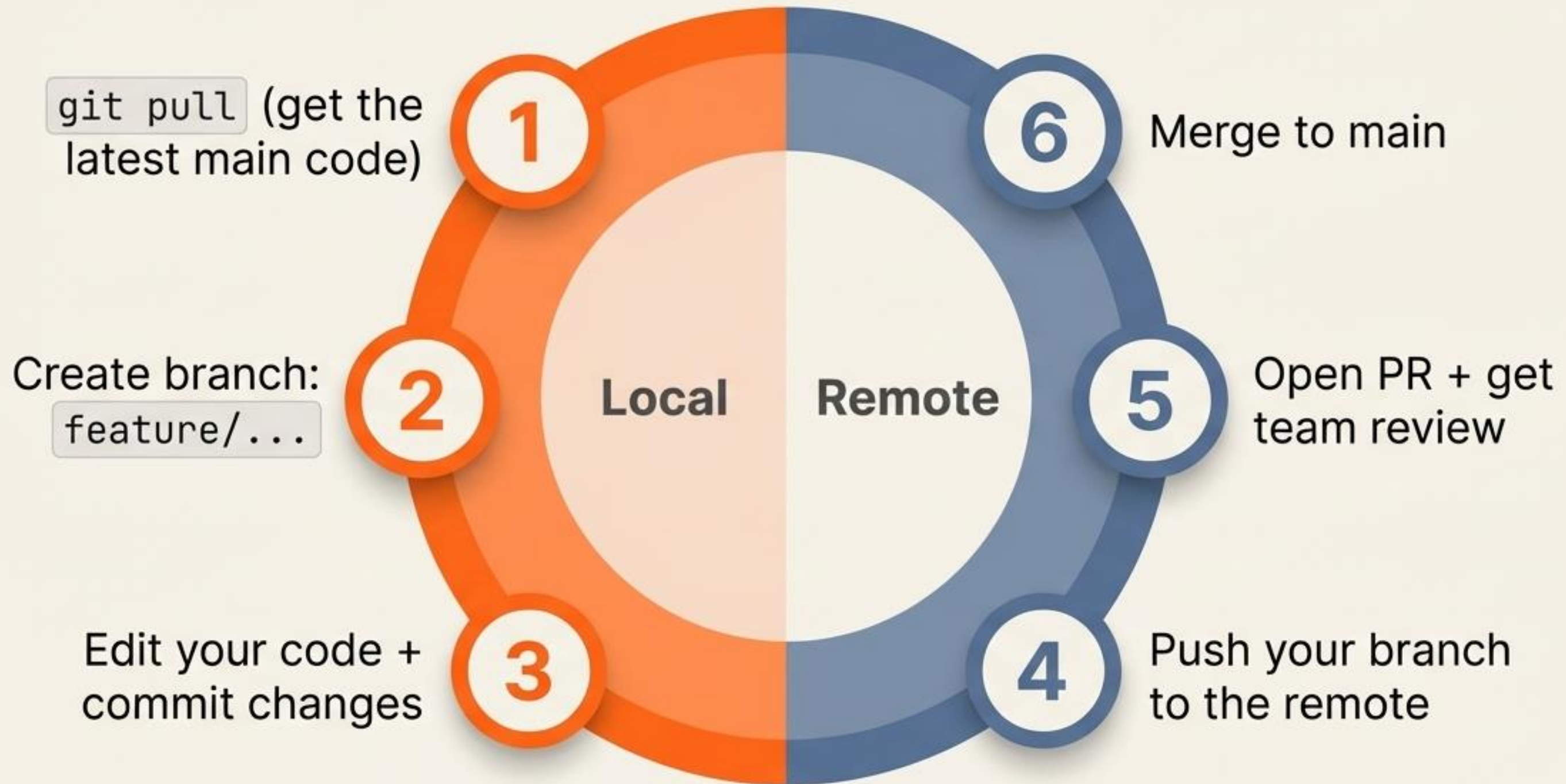
Pull Request (PR): The process of reviewing and merging a feature branch into the main codebase.

The standard team collaboration workflow

The "Main" branch is sacred—it represents your stable, working code. All active work happens in isolated feature branches. The lifecycle moves seamlessly from pushing a branch, to opening a Pull Request, to team review, and finally merging into main.



Six steps from starting work to shipping features



The essential Git toolkit

```
git clone <url>
```

Download a repository to your local machine.

```
git add <file>
```

Stage a file to be included in the next commit.

```
git pull
```

Fetch and merge the latest remote updates.

```
git checkout -b feature/name
```

Create and switch to a brand new branch.

```
git status
```

Check the current state of files and branches.

```
git commit -m "message"
```

Save a snapshot of staged changes with a note.

```
git push
```

Upload your local commits to the remote repository.

```
git merge main
```

Combine the main branch into your current branch.

Golden rules for professional engineering

Commit often

Focus on small, meaningful commits rather than massive, unreadable dumps of code.

Always pull before you push

Keep your local workspace synchronized to avoid headaches and merge conflicts.

Use clear messages



Fix data leakage in scaler.



Add confusion matrix plot.



The strict rule for AI repositories

Never push large data files or compiled models to the shared repository. Always use `.gitignore` to block these files from version control entirely.

The pre-Pull Request checklist

Before opening a PR, ensure:

- ☐ Branch name is correct and descriptive
- ☐ The code actually runs locally
- ☐ You have pulled the latest main branch
- ☐ All commits have good, readable messages
- ☐ No large data files are accidentally committed
- ☐ The PR description clearly explains what you changed

Saving your local progress

What is the main purpose of a commit in Git?

Answer: Save a snapshot of changes with a message.

Which command stages a file so it will be included in the next commit?

Answer: `git add <file>`

You made changes and want to save them locally. Which command is correct?

Answer: `git commit -m "message"`

Starting a new task safely

Your team policy says: Never commit directly to main. What should you do first when starting a new task?

Answer: `git checkout -b feature/task-name`

Which command shows the current branch and changed files?

Answer: `git status`

KNOWLEDGE CHECK

Syncing with the remote

What exactly does git pull do?

Answer: Fetches updates from remote and merges into your current branch.

You want to upload your branch to GitHub for the first time and set upstream. Which command is best?

Answer: `git push -u origin feature/my-branch`



KNOWLEDGE CHECK

Handling merge conflicts



A merge conflict usually happens when:

Answer: Two people change the same lines in the same file.

After resolving a merge conflict in a file, what is the correct next step?

Answer: `git add <conflicted_file>` then `git commit`

The final gate of collaboration

Why is a Pull Request (PR) important in team collaboration?



Answer: It allows code review and checks before merging into main.



- 1. Never lose work.**
- 2. Never overwrite teammate work.**

THANK
you